```
Output:
Uncaught TypeError: breakMyArray(): Return value must be of type array,
none returned
```

Ah, that's a hard no. Specifying a return type but not having a `return` statement in the function is not possible unless that type is `void`. Perhaps passing references around is something to be treated with caution? Something to think about!

Functions that return by reference also have a gotcha. Remember that `void` returning functions will always implicitly return `null`? What if we had a function that returns `null` by reference? (Note: if you're not sure what `static` means, we cover its behavior in more detail when discussing the concept of *scope* in a later section.)

```
function &getNull(): void
{
    static $val = null;
    return $val;
}

$ref = &getNull();
```

```
Output:
Deprecated: Returning by reference from a void function is deprecated
Fatal error: A void function must not return a value
```

Not one but two errors! Depending on which version of PHP 8 you are running you'll get the deprecation notice (returning by reference from a `void` function was deprecated in PHP 8.1) followed by a Fatal error. `void` functions will implicitly return `null` but an explicit `return` of a reference to `null` is not the same thing, so the `void` declaration is illegal. From PHP 8.2 onwards `null` is allowed as a standalone type, so the following declaration would be valid: `function &getNull(): null` (but would otherwise emit another fatal error for PHP <= 8.1).

Advanced Arguments

We've looked at the basics of passing data into functions, now let's build on that a little.